

```

.data
    msg1: .string "Enter your roll number: "
    msg2: .string "The sum from 1 to your roll number is: "

.text
.globl main

main:
    # Print input statement
    li a7, 4
    la a0, msg1
    ecall

    # Save input number
    li a7, 5
    ecall
    mv t0, a0

    li t1, 1
    li t2, 0

LOOP:
    bgt t1, t0, EXIT

    add t2, t2, t1 # sum = sum + i
    addi t1, t1, 1 # i = i + 1

    j LOOP

```

```

EXIT:
    li a7, 4
    la a0, msg2
    ecall

    mv a0, t2
    li a7, 1
    ecall

    li a7, 10
    ecall

```

Enter your roll number: 38  
The sum from 1 to your roll number is: 741

```

.data
    n: .string "Enter sum of your roll number (n): "
    ans: .string "The factorial is: "

.text
.globl main

main:
    # Display input message
    li a7, 4
    la a0, n
    ecall

    # Input integer
    li a7, 5
    ecall
    mv t1, a0 # factorial condition

    # t1 is our counter
    li t2, 1 # Answer storing
    li t3, 1

LOOP:
    blt t1, t3, EXIT

    mul t2, t2, t1
    addi t1, t1, -1

    j LOOP

```

```

EXIT:
    li a7, 4
    la a0, ans
    ecall

    mv a0, t2
    li a7, 1
    ecall

    li a7, 10
    ecall

```

Enter sum of your roll number (n) : 11  
The factorial is: 39916800

```

.data
    n: .string "Enter some integer "
    composite: .string "The number is composite"
    prime: .string "The number is prime"
    neither: .string "The number is neither prime nor composite"

.text
.globl main

main:
    # Display input message
    li a7, 4
    la a0, n
    ecall

    # Input integer
    li a7, 5
    ecall
    mv t0, a0      # factorial condition

    li t1, 2      # Initiate for loop and check
    blt t0, t1, IS_NEITHER

LOOP:
    mul t2, t1, t1      # t2 = i * i
    bgt t2, t0, IS_PRIME # t2 > n = PRIME

    rem t3, t0, t1      # t3 = n % i
    beq t3, zero, IS_COMPOSITE

    addi t1, t1, 1      # i++
    j LOOP

IS_PRIME:
    li, a7, 4
    la a0, prime
    ecall
    j EXIT

IS_COMPOSITE:
    li, a7, 4
    la a0, composite
    ecall
    j EXIT

IS_NEITHER:
    li, a7, 4
    la a0, neither
    ecall

EXIT:
    li a7, 10
    ecall

```

Enter some integer 37  
The number is prime

Enter some integer 38  
The number is composite

```

.data
    msg1: .asciz "Enter your roll number: "
    msg2: .asciz "Enter your friend's roll number: "
    result: .asciz "The approximate average is: "

.text
.globl main

main:
    # Get your roll number
    li a7, 4
    la a0, msg1
    ecall

    li a7, 5
    ecall
    mv s0, a0

    # Get friend's roll number
    li a7, 4
    la a0, msg2
    ecall

    li a7, 5
    ecall
    mv s1, a0

    mv a0, s0      # Parameter 1: Your roll number
    mv a1, s1      # Parameter 2: Friend's roll number
    li a2, 3       # Parameter 3: The number 3

    #AVERAGE procedure call
    jal AVERAGE

    #Store returned result
    mv s2, a0

    # 6. Display result
    li a7, 4
    la a0, result
    ecall

    mv a0, s2
    li a7, 1
    ecall

    # 7. Exit
    li a7, 10
    ecall

AVERAGE:
    # Sum the three parameters
    add t0, a0, a1
    add t0, t0, a2

    div a0, t0, a2

    ret

```

Enter your roll number: 38  
Enter your friend's roll number: 39  
The approximate average is: 26

```

data
    num: .asciz "Enter the base (Sum of your roll number): "
    pow: .asciz "Enter the power (0-9): "
    result: .asciz "The result of num^pow is: "

text
globl main

main:
    #Input for num
    li a7, 4
    la a0, num
    ecall

    li a7, 5
    ecall
    mv s0, a0

    #Input for pow
    li a7, 4
    la a0, pow
    ecall
    li a7, 5
    ecall
    mv s1, a0

    #POWER function
    mv a0, s0
    mv a1, s1
    jal POWER

    mv s2, a0          # Save result

                                #Result
                                li a7, 4
                                la a0, result
                                ecall

                                mv a0, s2
                                li a7, 1
                                ecall

                                li a7, 10
                                ecall

POWER:
    li t0, 1           #result storing
    li t1, 0           #counter

POWER_LOOP:
    beq t1, a1, EXIT

    mul t0, t0, a0     # result = result * base
    addi t1, t1, 1     # i++

    j POWER_LOOP

EXIT:
    mv a0, t0
    ret

```

```

Enter the base (Sum of your roll number): 11
Enter the power (0-9): 2
The result of num^pow is: 121

```